

Advanced QEMU/KVM Networking on Ubuntu – Michael Waterman

 michaelwaterman.nl/2023/12/12/advanced-netplan-config-on-ubuntu

Michael Waterman

December 12, 2023



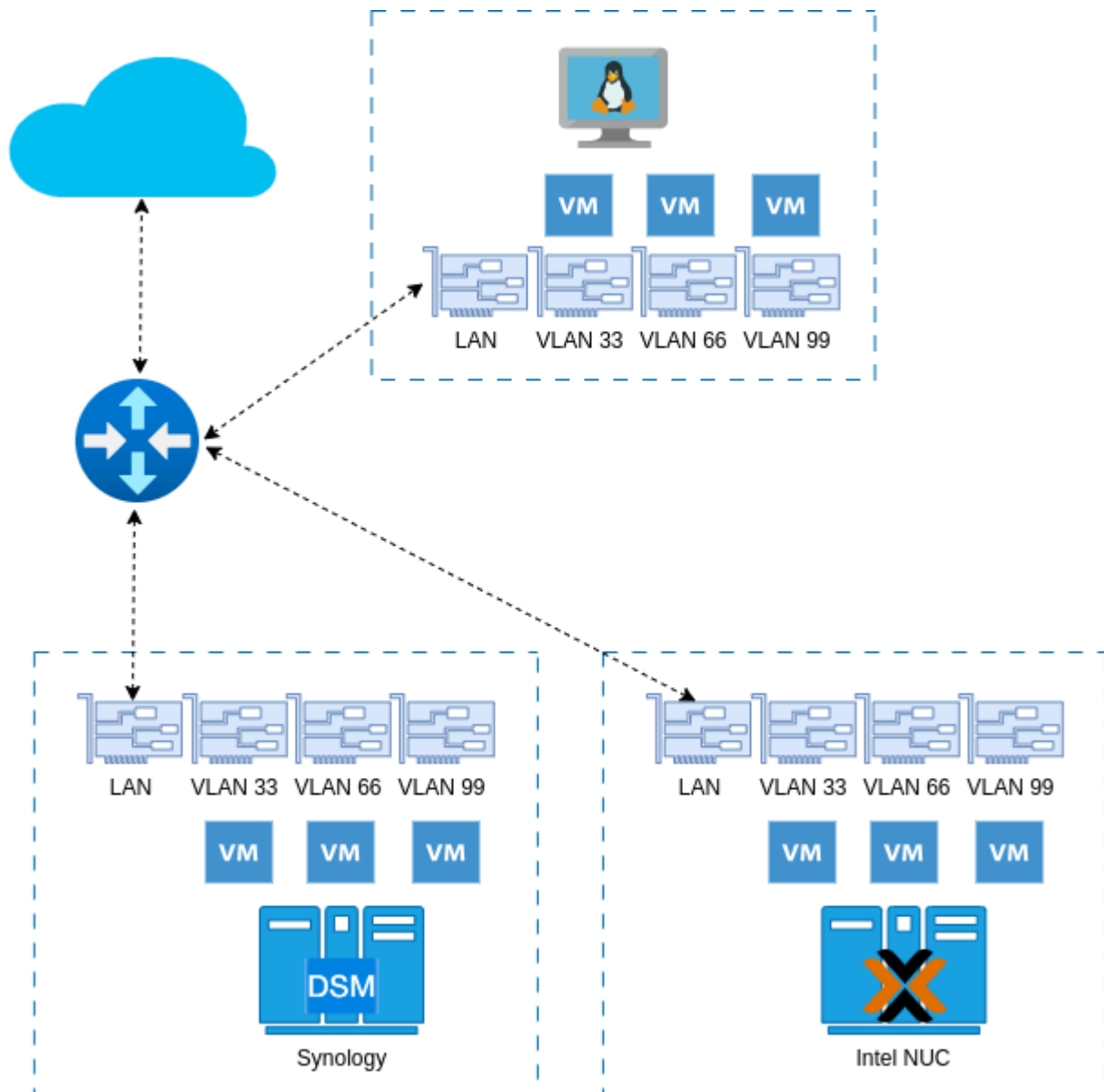
Transitioning between Operating Systems can be a challenge. Many aspects of what you're used to work differently and you should expect a learning curve.

Well, that was a weird intro for a blog post that has Linux networking in the title! Actually I'm saying goodbye to Windows as my primary system.... yes you read that correctly. It's not that I don't like the system anymore, it's the direction Microsoft is taking with AI and the integration into the OS that made me take this decision.

I'm just not keen on having some intelligence (besides me) controlling my machine, besides I'm one of those people that are very reserved in using AI all together, it just seem to go too fast, too unregulated and we're simply not prepared. Doomsday aside, I'm saying goodbye to Windows and hello to Ubuntu Linux, at the time of writing version 23.10.1.

So I did have a couple of requirements for making the transition, one of them being a decent hypervisor for running my labs. Now I do have a bit of an emulated homelab running. Parts are running on my Synology server, the main bulk of servers run on a Intel NUC and my primary PC is used to run the VM's that I need to figure out stuff, like malware analysis, pen testing or blue team configuration testing. For my new setup on Ubuntu I've decided on the Hypervisor QEMU/KVM, which works like a charm.

In my LAB setup I've configured several VLANs to separate the networks and just as in real life (at least hopefully) put firewalls in between. As I have a home network, I separate network traffic by putting a VLAN tag on the NIC of the VM and have them communicate by tag over the network.



Having Proxmox on the Intel NUC, it's easy as defining the VLAN ID on the VM virtual NIC and off it goes, same with Hyper-V when I was still running Windows 11.

During my initial testing I couldn't find a setting in QEMU/KVM Virtual Machine Manager to set a VLAN tag. Apparently it works a bit differently, and it turned out that I needed to create a network bridge and connect a VM to that bridge. Next step was to define a VLAN tag on that bridge, so when a VM would connect to that bridge it would automatically use the correct VLAN. After many tries, I came up with the following configuration using netplan:

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
```

```

renderer: NetworkManager
ethernets:
  eno1: # Replace with the name of your NIC
    dhcp4: no
    dhcp6: no
    mtu: 9000
bridges:
  br0:
    interfaces: [eno1]
    dhcp4: yes
    dhcp6: no
  br33:
    interfaces: [br0.33]
    dhcp4: no
    dhcp6: no
  br66:
    interfaces: [br0.66]
    dhcp4: no
    dhcp6: no
  br99:
    interfaces: [br0.99]
    dhcp4: no
    dhcp6: no
vlans:
  br0.33:
    link: br0
    id: 33
  br0.66:
    link: br0
    id: 66
  br0.99:
    link: br0
    id: 99

```

In the code above “*eno1*” is my physical network interface, which you can enumerate with “*ip a*”. Under “bridges” there are 4 bridges defined, “*br0*”, “*br33*”, “*br66*” and “*br99*”. It’s just a naming scheme that I use, you can basically name them whatever you want. The bridges that need to use a VLAN connect to their respective VLAN interfaces as defined in the “*vlans*” section. These in turn are attached to the default bridge “*br0*”. “*br0*” is the interface that replaces the configuration on the physical NIC, my PC now connects to that bridge to get to the network/Internet. This is the only interface that gets an IP from my DHCP server, as this is the only interface that my PC is going to use to connect to the network. You could provide an IP to the bridges however! This would allow your PC to also use a specific bridge and associated VLAN to directly connect to the Virtual Machines on that network, really cool stuff.

To use this configuration, open up a terminal and paste the code into the “*01-network-manager-all.yaml*” file in “*etc/netplan*”. Execute “*sudo netplan apply*” after saving the file.

Note! You will temporally lose network connectivity while the systems creates the bridges and VLANs.

```
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc pfifo_fast master br0 state UP group default qlen 1000
   link/ether d8:5e:d3:ab:82:16 brd ff:ff:ff:ff:ff:ff
   altname enp0s31f6
3: br33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue state UP group default qlen 1000
   link/ether 72:e5:3a:a9:f7:6b brd ff:ff:ff:ff:ff:ff
   inet 169.254.102.26/16 brd 169.254.255.255 scope link noprefixroute br33
       valid_lft forever preferred_lft forever
   inet6 fe80::70e5:3aff:fea9:f76b/64 scope link
       valid_lft forever preferred_lft forever
4: br66: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue state UP group default qlen 1000
   link/ether ee:40:27:83:21:42 brd ff:ff:ff:ff:ff:ff
   inet 169.254.203.123/16 brd 169.254.255.255 scope link noprefixroute br66
       valid_lft forever preferred_lft forever
   inet6 fe80::ec40:27ff:fe83:2142/64 scope link
       valid_lft forever preferred_lft forever
5: br99: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue state UP group default qlen 1000
   link/ether be:18:8e:76:20:0c brd ff:ff:ff:ff:ff:ff
   inet 169.254.108.234/16 brd 169.254.255.255 scope link noprefixroute br99
       valid_lft forever preferred_lft forever
   inet6 fe80::bc18:8eff:fe76:200c/64 scope link
       valid_lft forever preferred_lft forever
6: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue state UP group default qlen 1000
   link/ether e2:dc:fd:4b:59:2a brd ff:ff:ff:ff:ff:ff
   inet 192.168.10.18/24 brd 192.168.10.255 scope global dynamic noprefixroute br0
       valid_lft 18703sec preferred_lft 18703sec
   inet6 fe80::e0dc:fdff:fe4b:592a/64 scope link
       valid_lft forever preferred_lft forever
7: br0.99@br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue master br99 state UP group default qlen 1000
   link/ether e2:dc:fd:4b:59:2a brd ff:ff:ff:ff:ff:ff
8: br0.66@br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue master br66 state UP group default qlen 1000
   link/ether e2:dc:fd:4b:59:2a brd ff:ff:ff:ff:ff:ff
9: br0.33@br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc noqueue master br33 state UP group default qlen 1000
   link/ether e2:dc:fd:4b:59:2a brd ff:ff:ff:ff:ff:ff
10: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
   link/ether 52:54:00:95:24:16 brd ff:ff:ff:ff:ff:ff
   inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
       valid_lft forever preferred_lft forever
```

Note! I've initially tested this on Ubuntu running as a VM in Hyper-V and VMWare workstation. Both ended up in epic failures, well to be fair Hyper-V was the largest failure where the entire system would break, at least VMWare created a single bridge but eventually also broke. So moral of this story is, use this only on real hardware.

More info on configuration options for netplan can be located [here](#).

Assigning the bridge and VLAN

The next step would be to configure the Virtual Machine to use the correct virtual bridge. It's really easy to do that within QEMU/KVM, just start the wizard and select the configuration options, until you reach the final page, "Step 5 of 5". Expand the "Network selection", make sure the "Bridge device" is selected and enter the ID of the bridge the VM needs to communicate over. In the example below it's "br66".

New VM

Create a new virtual machine
Step 5 of 5

Ready to begin the installation

Name: win10-2

OS: Microsoft Windows 10

Install: Local CDROM/ISO

Memory: 4096 MiB

CPUs: 2

Storage: 40.0 GiB /var/lib/libvirt/images/win10-2.qcow2

☐ Customize configuration before install

Network selection

Bridge device...

Device name: br66

Cancel Back Finish

Finally click “*Finish*” and see the magic happen! Hopefully this helps someone out there!

As always, let me know if you have any feedback or remarks.